

Centro Visite

Progetto di Ingegneria del Software – Parte B

Lorenzo Dolzanelli MAT 740643

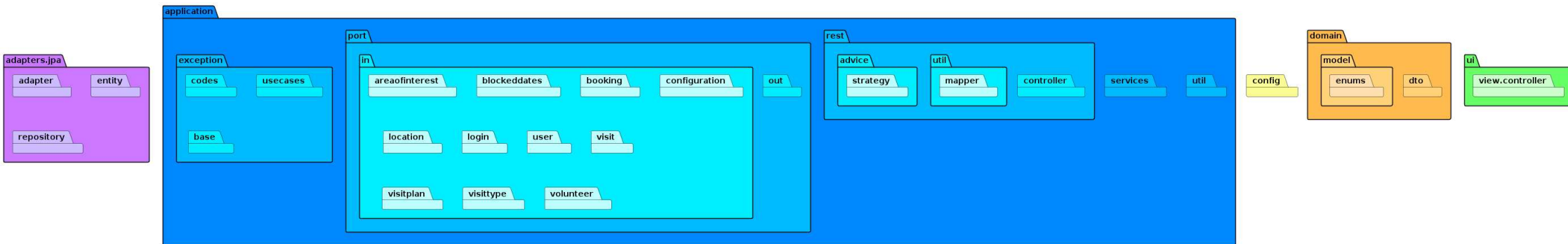
Khater Abdelaziz MAT 741614

Daniele Miguel Franchi MAT 743230

Indice dei contenuti

A. Separazione modello-vista	3-5
B. Esempi di pattern GRASP	
A. GRASP 1 – Information Expert	6-7
B. GRASP 2 – Controller	8-10
C. Esempi di pattern SOLID	
A. SOLID 1 – Dependency Inversion	11
B. SOLID 2 – Interface Segregation	12
D. Esempi di pattern GoF	
A. GoF 1 – Strategy	13-15
B. GoF 2 – Adapter	16-17
E. Esempi di testing	18-22
F. Esempi di refactoring	24-28

Diagramma dei package



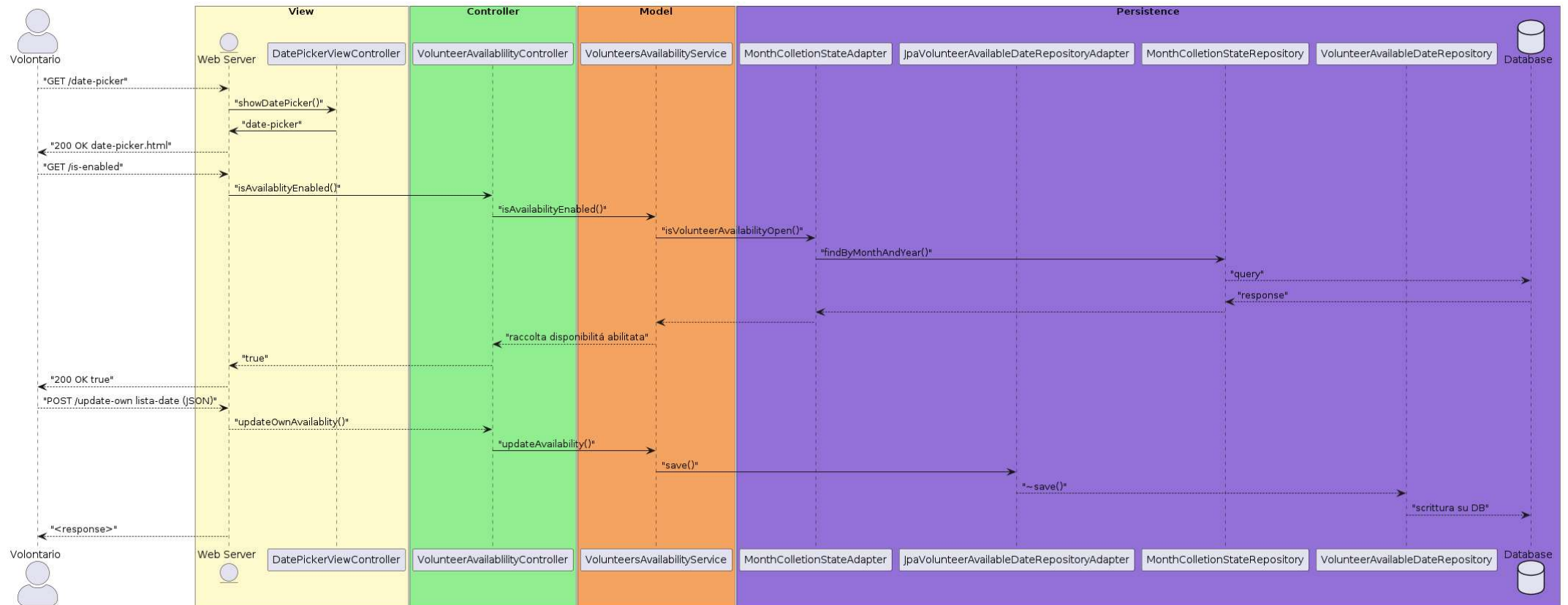
Separazione modello-vista

La separazione avviene seguendo il pattern MVC:

- la **View** é contenuta nei template HTML in Thymeleaf.
- I **Controller** sono i REST Controller, che espongono le API per l'invio e il recupero dei dati dal backend.
- Il **Model** è costituito dai Service che svolgono la logica di business, dal layer di persistence che si interfaccia con il DB tramite le repository.

Esempio con SSD

Disponibilità dei volontari



GRASP 1 – information expert

PRIMA

```
// VisitPlanService
```

```
private List<VolunteerAvailableDate> initMonthAvailabilities(List<User> volunteers, int month, int year) {  
    List<VolunteerAvailableDate> monthAvailabilities = new ArrayList<>();  
    for (User v : volunteers) {  
        // per ogni volontario recupera le sue disponibilità e seleziona quelle del mese  
        // selezionato  
        List<VolunteerAvailableDate> volunteerAvailabilities = new ArrayList<>();  
  
        volunteerAvailabilityRepository.findById(v.getId()).stream().filter(  
            va -> va.getAvailableDate().getMonthValue() == month && va.getAvailableDate().getYear() == year  
        ).forEach(volunteerAvailabilities::add);  
    }  
}
```

Nella classe `VisitPlanService` il metodo `initMonthAvailabilities()` chiamava diverse repository per selezionare più dati, che andava poi a gestire e associare per recuperare informazioni su dati del database. Si é deciso di spostare questo compito alla classe `JpaVolunteerRepositoryAdapter` visto che fornisce già funzioni di base per la lettura/scrittura dati da DB.

GRASP 1 – information expert

DOPO

application.port.out.VolunteerAvailableDateRepositoryPort

```
public interface VolunteerAvailableDateRepositoryPort {  
    void save(VolunteerAvailableDate availableDate);  
    void delete(VolunteerAvailableDate availableDate);  
    boolean exists(int volunteerId, LocalDate date);  
    List<VolunteerAvailableDate> findByVolunteerId(int volunteerId);  
    List<VolunteerAvailableDate> findByDate(LocalDate date);  
+    List<VolunteerAvailableDate> findByYearMonth(YearMonth yearMonth);  
}
```

adapters.jpa.adapter.JpaVolunteerAvailableDateRepositoryAdapter

```
@Override  
public List<VolunteerAvailableDate> findByYearMonth(YearMonth yearMonth) {  
    return repository.findAll()  
        .stream()  
        .filter(entity -> entity.getId().getAvailableDate().getYear() == yearMonth.getYear())  
        .filter(entity -> entity.getId().getAvailableDate().getMonth() == yearMonth.getMonth())  
        .map(this::toDomain)  
        .collect(Collectors.toList());  
}
```

VisitPlanService#createVisitPlan()

```
...  
YearMonth nextMonth = YearMonth.from(LocalDate.now(clock)).plusMonths(1);  
  
List<VolunteerAvailableDate> volunteerAvailabilitiesNextMonth = volunteerAvailabilityRepository.findByYearMonth(nextMonth);  
...
```

GRASP 2 - Controller

Il layer di REST Controller espone API da lui definite, che vengono chiamate dalla View. Una volta che riceve la chiamata, il Controller delega lo svolgimento della operazione al layer dei Service, che si occupano di applicare la logica di business. Viene poi fornita la risposta ricevuta all'utente che ha effettuato la richiesta.

GRASP 2 - Controller

Esempio chiamata GET da BookingController

```
@GetMapping("/booking-details/{bookingCode}") Abdelaziz Khater +2 *
public ResponseEntity<BookingDetailsDTO> bookingDetails(@PathVariable String bookingCode) {
    Booking booking = bookingQueryService.getBookingByCode(bookingCode);
    Visit visit = bookingQueryService.getVisitByBookingCode(bookingCode);
    String locationName = locationQueryService.getLocationForVisitType(visit.getVisitType()).getName();

    var dto = new BookingDetailsDTO(booking.getUser().getId(), visit.getId(), booking.getUser().getNickname(),
        booking.getVisitorsNames(), visit.getVisitType().getTitle(), locationName,
        visit.getDate().format(DateTimeFormatter.ofPattern("dd/MM/yyyy")));

    return ResponseEntity.ok(dto);
}
```

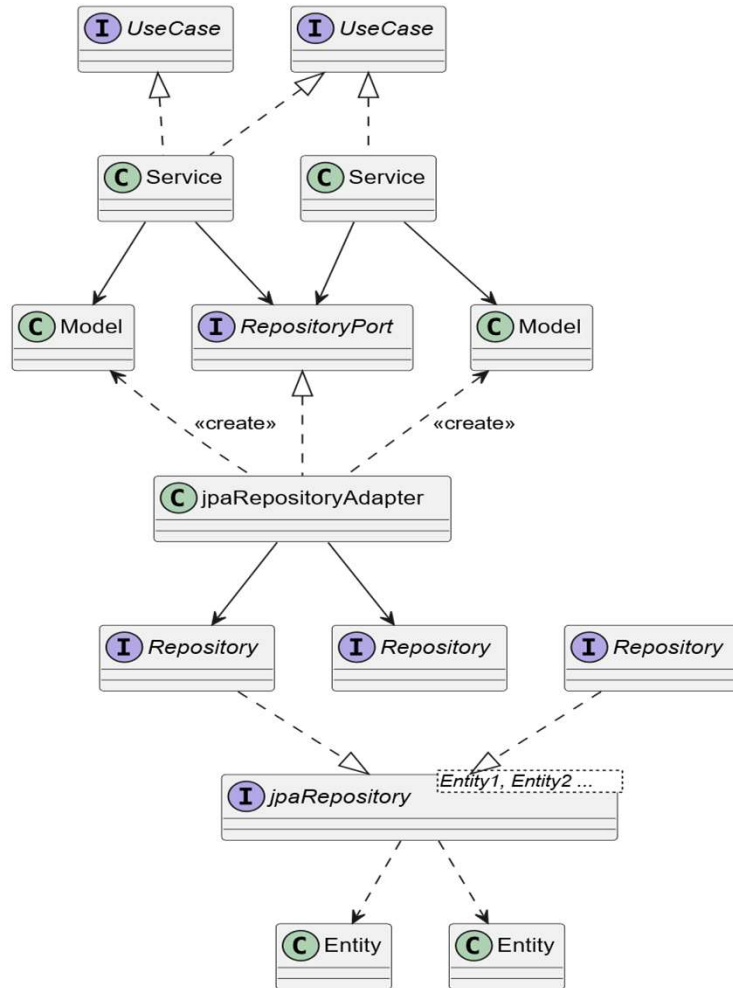
GRASP 2 - Controller

Esempio chiamata POST da BookingController

```
@PostMapping("/book") * LDolzanelli +1 *
public ResponseEntity<String> bookVisit(@RequestBody BookingRequestDTO dto) {
    Visit visit = visitService.getVisitById(dto.visitId());
    User user = userService.findById(dto.userId()) //
        .orElseThrow(() -> new IllegalArgumentException("User with id " + dto.userId() + " not found"));

    createBookingService.bookVisit(visit, user, dto.visitorsNames());
    return ResponseEntity.ok(body: "Prenotazione effettuata con successo");
}
```

SOLID 1 – dependency inversion



L'interazione tra le classi dei diversi strati (dominio, persistenza) avviene mediante delle interfacce che costituiscono un'astrazione delle implementazioni.

In questo modo si evita di avere una dipendenza diretta tra una classe e l'altra. Un esempio sono le interfacce del package `RepositoryPort`.

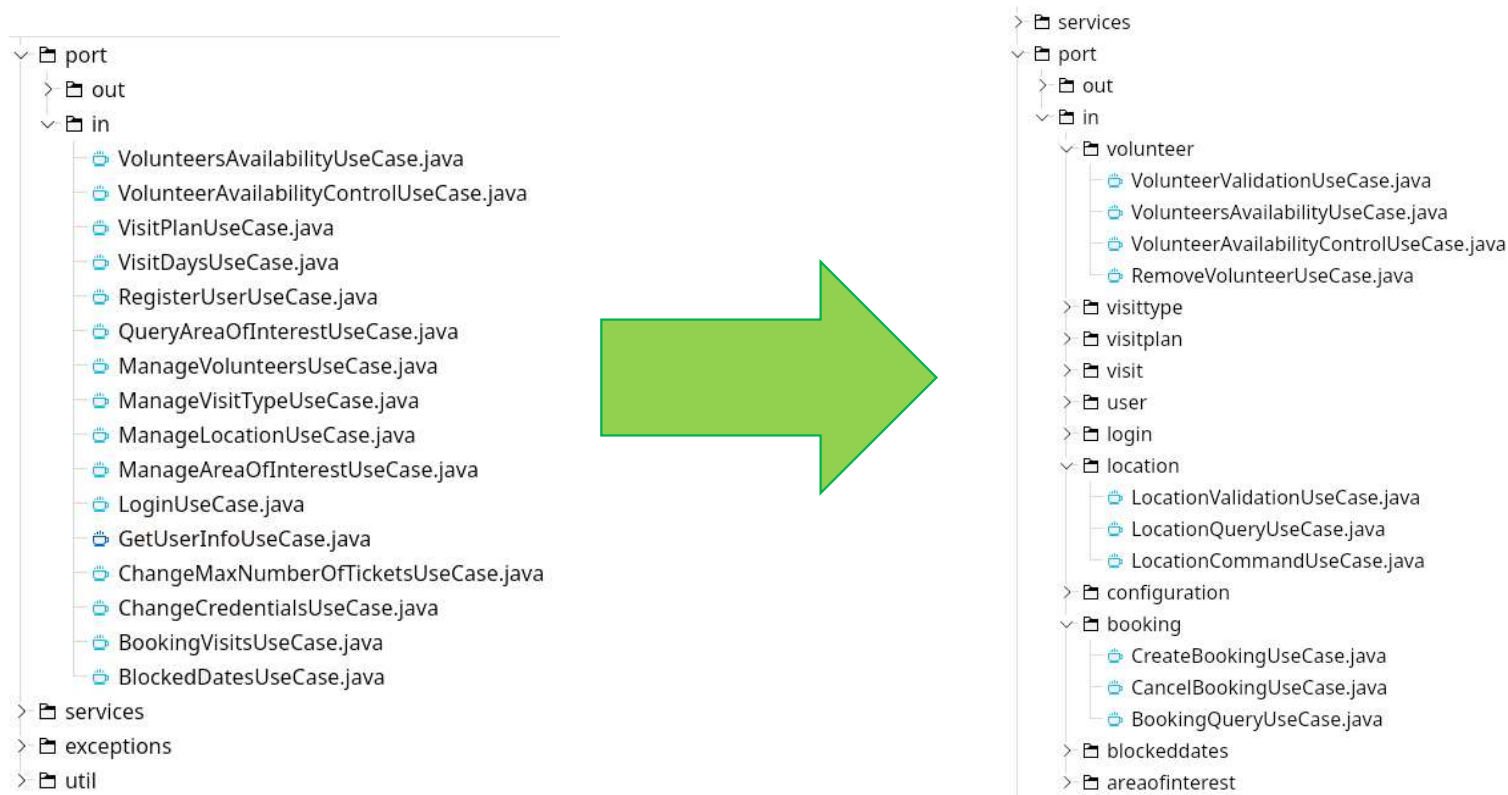
Le `JpaRepositoryAdapter`, chiamate tramite le `RepositoryPort`, producono oggetti `Model` utilizzabili dai `Services`.

Gli oggetti `Model` vengono prodotti a partire dagli oggetti `Entity` recuperati tramite le `Repository`.

SOLID 2 – Interface segregation

Riorganizzazione delle interfacce UseCase, implementate nei Service.

Le interfacce contenevano troppe funzioni, di conseguenza sono state scorporate in interfacce più piccole che svolgono compiti precisi



GoF 1 – Strategy

Per ogni caso d'uso è stata definita una classe di eccezione dedicata che estende una base comune.

La classe `BaseUseCaseException` fornisce un'interfaccia uniforme per identificare il tipo di caso d'uso e il relativo codice di errore.

Ogni caso d'uso (es. booking, visit, user, ecc.) definisce la propria eccezione specializzata.

```
public abstract class BaseUseCaseException extends RuntimeException {  
    public abstract String getUseCaseType();  
    public abstract Object getErrorCode();  
}
```

```
public class BookingException extends BaseUseCaseException {  
    private static final String USE_CASE_TYPE = "booking";  
    private final BookingErrorCode errorCode;  
    ...  
}
```

L'architettura utilizza il pattern Strategy per delegare la gestione di ogni eccezione alla strategia corrispondente.

`GlobalExceptionHandler` individua la strategia appropriata e ne invoca il metodo `handleException()`.

```
public class GlobalExceptionHandler {  
    @ExceptionHandler(BaseUseCaseException.class)  
    public ResponseEntity<String> handleUseCaseException(BaseUseCaseException ex) {  
        ExceptionHandlingStrategy strategy = findStrategy(ex);  
        if (strategy != null) {  
            return strategy.handleException(ex);  
        }  
    }  
    private ExceptionHandlingStrategy findStrategy(Exception exception) {  
        return strategies.stream().filter(strategy -> strategy.canHandle(exception)).  
            findFirst().orElse(null);  
    }  
}
```



In base al codice d'errore, la strategia seleziona lo status HTTP più appropriato (es. 400, 404) e costruisce un messaggio chiaro e non tecnico per l'utente. I dettagli interni vengono registrati nei log di sistema; al client viene restituita solo l'informazione utile all'azione successiva.

```
public class BookingExceptionHandlerStrategy implements ExceptionHandlingStrategy
{
    @Override
    public ResponseEntity<String> handleException(Exception exception) {
        String userMessage = translateErrorCode(bookingException.getErrorCode());
        HttpStatus status = determineHttpStatus(bookingException.getErrorCode());
        return ResponseEntity.status(status).body(userMessage);
    }
    private String translateErrorCode(BookingErrorCode errorCode) {
        return switch (errorCode) {
            case BOOKING_NOT_FOUND -> "Prenotazione non trovata";
            case BOOKING_NOT_CANCELABLE -> "La prenotazione non può essere annulla
ta";
            case USER_NOT_AUTHORIZED -> "Non puoi annullare questa prenotazione per
ché non sei il proprietario.";
            case INVALID_INPUT_DATA -> "Dati di input non validi";
            case NOT_ENOUGH_SEATS -> "I posti disponibili sono insufficienti per la
tua richiesta";
        };
    }
    private HttpStatus determineHttpStatus(BookingErrorCode errorCode) {
        return switch (errorCode) {
            case BOOKING_NOT_FOUND -> HttpStatus.NOT_FOUND;
            case BOOKING_NOT_CANCELABLE -> HttpStatus.CONFLICT;
            case USER_NOT_AUTHORIZED -> HttpStatus.FORBIDDEN;
            case INVALID_INPUT_DATA -> HttpStatus.BAD_REQUEST;
            case NOT_ENOUGH_SEATS -> HttpStatus.CONFLICT;
        };
    }
}
```

GoF 2 – Adapter

Il pattern Adapter è stato applicato alle Repository JPA fornite da Spring Data per introdurre un'interfaccia stabile (rappresentata da `RepositoryPort`) che permette ai Service di interagire con il livello di persistenza in modo naturale e indipendente dai dettagli di implementazione.

L'Adapter ha il compito di trasformare i dati tra il modello di dominio e le Entity del database. In lettura, converte le Entity in oggetti di dominio, rendendo i dati utilizzabili dai Service. In scrittura, effettua la conversione inversa.

Target

```
public interface BookingRepositoryPort { 7 usages 1 implementation AbdelazizKhater *
    void save(Booking booking, int visitId); 1 implementation AbdelazizKhater
    Optional<Booking> findByBookingCode(String bookingCode); 3 usages 1 implementation AbdelazizKhater
    List<Booking> findAll(); 1 implementation AbdelazizKhater
    List<Booking> findAllByVisitId(int visitId); 1 usage 1 implementation AbdelazizKhater
    List<Booking> findAllByUserId(int userId); 1 usage 1 implementation AbdelazizKhater
}
```

Adapter

```
public class JpaBookingAdapter implements BookingRepositoryPort {
    private final BookingRepository bookingRepository;
    private final UserRepository userRepository;
    private final VisitRepository visitRepository;

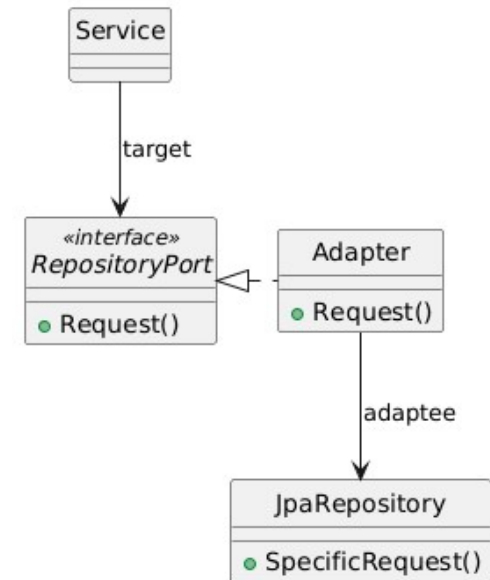
    @Override AbdelazizKhater
    public void save(Booking booking, int visitId) {
        List<BookingEntity> entities = toEntity(booking, visitId);
        bookingRepository.saveAll(entities);
    }
}
```

Adaptee

```
@Repository 4 usages AbdelazizKhater *
public interface BookingRepository extends JpaRepository<BookingEntity, BookingId> {
    List<BookingEntity> findByIdBookingCode(String bookingCode); 3 usages AbdelazizKhater

    List<BookingEntity> findByVisit_Id(int visitId); 1 usage AbdelazizKhater

    List<BookingEntity> findByUser_Id(int userId); 1 usage AbdelazizKhater
}
```



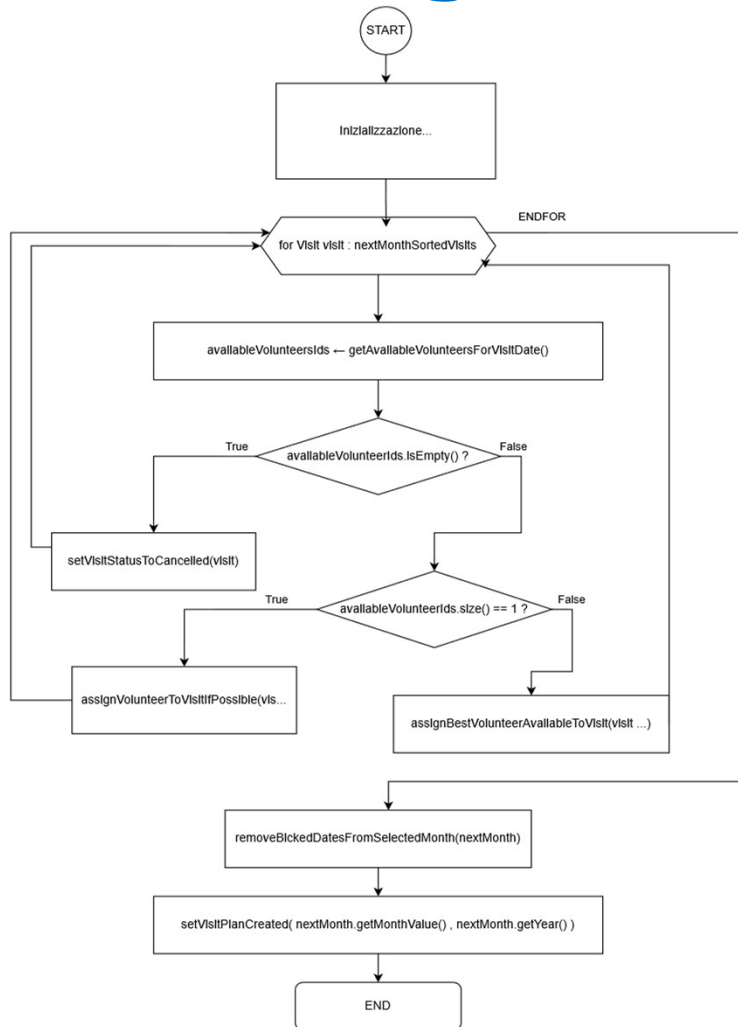
Testing White Box – createVisitPlan()

Data l'esigenza di eseguire piú refactoring sul metodo `createVisitPlan()` in `VisitPlanService` (vedi a slide 23), prima di operare é stato necessario definire diversi casi di test in modo da verificare che l'algoritmo si comportasse sempre alla stessa maniera.

Di seguito il grafo che descrive il funzionamento generale dell'algoritmo con i casi di test individuati

Testing White Box – createVisitPlan()

createVisitPlan()



```
public void createVisitPlan() {
    if (!canCreateVisitPlan()) {
        throw new VisitPlanException(VisitPlanErrorCode.CANT_BE_CREATED,
            "Non è possibile creare il piano di visita per il mese successivo.");
    }

    YearMonth nextMonth = YearMonth.from(LocalDate.now(clock)).plusMonths(monthsToAdd: 1);

    List<VolunteerAvailableDate> volunteerAvailabilitiesNextMonth = volunteerAvailabilityRepository
        .findByYearMonth(nextMonth);

    List<Visit> nextMonthSortedVisits = getNextMonthVisitsSorted(nextMonth.getMonthValue(), nextMonth.getYear());

    List<VolunteerIdWithVisitCount> volunteerIdWithVisitCounts = initializeVolunteerWithVisitCount();

    for (Visit visit : nextMonthSortedVisits) {
        List<Integer> availableVolunteerIds = getAvailableVolunteersForVisitDate(visit,
            volunteerAvailabilitiesNextMonth);

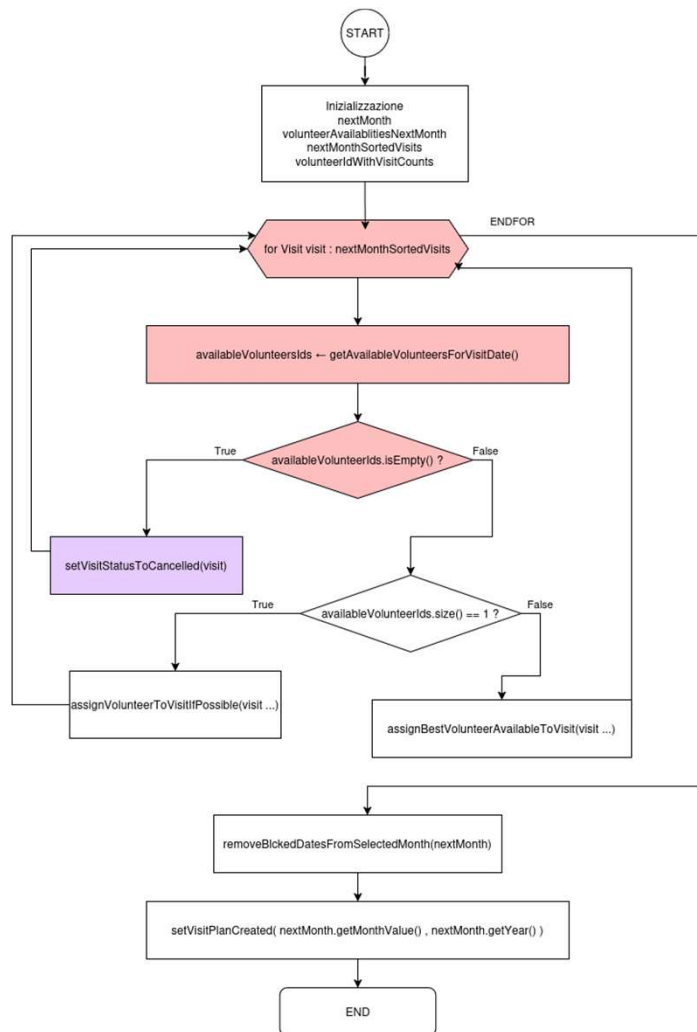
        if (availableVolunteerIds.isEmpty()) {
            setVisitStatusToCancelled(visit);
        } else if (availableVolunteerIds.size() == 1) {
            assignAvailableVolunteerToVisitIfPossible(visit, availableVolunteerIds,
                volunteerAvailabilitiesNextMonth, volunteerIdWithVisitCounts);
        } else {
            assignBestVolunteerAvailableToVisit(visit, availableVolunteerIds, volunteerIdWithVisitCounts,
                volunteerAvailabilitiesNextMonth);
        }
    }

    removeBlockedDatesFromSelectedMonth(nextMonth);

    setVisitPlanCreated(nextMonth.getMonthValue(), nextMonth.getYear());

    statePort.setVisitPlanCreated(nextMonth.getMonthValue(), nextMonth.getYear(), created: true);
}
```

Testing White Box – createVisitPlan()



Nessuna disponibilità - visita cancellata

```
@Test @LDolzaneli
void createVisitPlan_NoVolunteersAvailable_ShouldCancelVisit() {
    User user = new User( id: 1, nickname: "", password: "", Role.VOLUNTEER, firstLogin: false);

    when(userInfoUseCase.getUsersByRole(Role.VOLUNTEER)).thenReturn(List.of(user));
    when(volunteerAvailabilityRepository.findByVolunteerId(1)).thenReturn(List.of());

    VisitType visitTypeMock = mock(VisitType.class);
    Visit visit = new Visit(LocalDate.of( year: 2025, month: 10, dayOfMonth: 5), user, //
        visitTypeMock, List.of(), VisitStatus.PROPOSED);

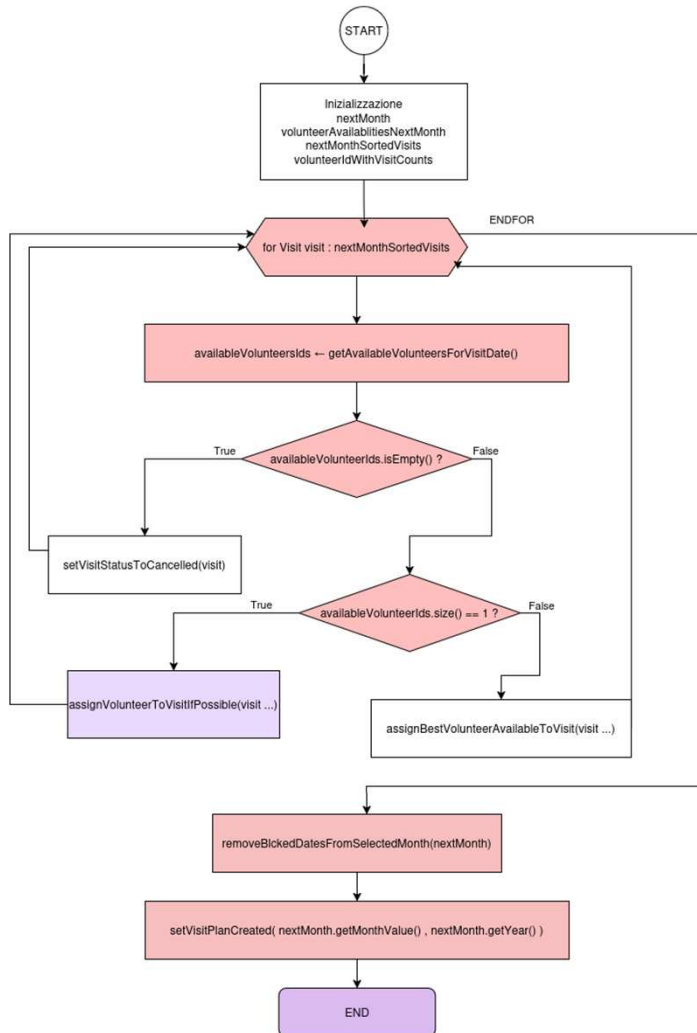
    when(visitRepository.findAll()).thenReturn(Set.of(visit));

    service.createVisitPlan();

    assertEquals(VisitStatus.CANCELLED, visit.getVisitStatus());
    verify(visitRepository).save(visit);
    verify(statePort).setVisitPlanCreated( month: 10, year: 2025, created: true);
}
```

Testing White Box – createVisitPlan()

Singola disponibilità - visita confermata



```
@Test
void createVisitPlan_OneVolunteerAvailableAndCanDoVisit_ShouldAssignVisit() {
    User volunteer = new User(id: 1, nickname: "volunteer", password: "test", Role.VOLUNTEER, firstLogin: false);
    when(userInfoUseCase.getUsersIdsByRole(Role.VOLUNTEER)).thenReturn(List.of(volunteer.getId()));
    when(userInfoUseCase.findById(1)).thenReturn(Optional.of(volunteer));

    LocalDate date = LocalDate.of(year: 2025, month: 10, dayOfMonth: 10);
    VolunteerAvailableDate availableDate = new VolunteerAvailableDate(volunteerId: 1, date);

    VisitType visitTypeMock = mock(VisitType.class);
    Visit visit = new Visit(date, volunteer: null, visitTypeMock, List.of(), VisitStatus.PROPOSED);
    when(visitRepository.findAll()).thenReturn(Set.of(visit));
    when(visitTypeRepository.findById(1)).thenReturn(Set.of(visitTypeMock));

    List<VolunteerAvailableDate> volunteerAvailableDates = new ArrayList<>();
    volunteerAvailableDates.add(availableDate);

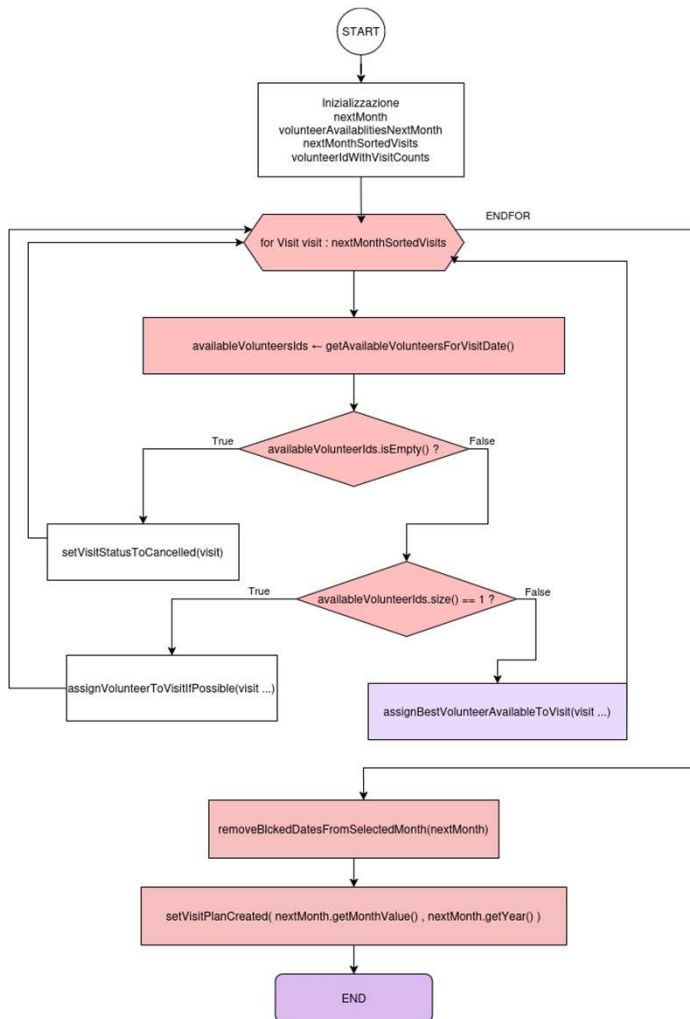
    when(volunteerAvailabilityRepository.findByYearMonth(YearMonth.of(year: 2025, month: 10)))
        .thenReturn(volunteerAvailableDates);

    service.createVisitPlan();

    assertEquals(VisitStatus.PROPOSED, visit.getVisitStatus());
    assertEquals(expected: 1, visit.getVolunteer().getId());
    verify(visitRepository).save(visit);
    verify(statePort).setVisitPlanCreated(month: 10, year: 2025, created: true);
}
```

Testing White Box – createVisitPlan()

Piú disponibilitá - scelta del volontario migliore



```
@Test
void createVisitPlan_MultipleVolunteersAvailable_ShouldAssignBestVolunteer() {
    User volunteer1 = new User( id: 1, nickname: "", password: "", Role.VOLUNTEER, firstLogin: false);
    User volunteer2 = new User( id: 2, nickname: "", password: "", Role.VOLUNTEER, firstLogin: false);
    when(userInfoUseCase.getUsersIdsByRole(Role.VOLUNTEER))
        .thenReturn(List.of(volunteer1.getId(), volunteer2.getId()));
    when(userInfoUseCase.findById(1)).thenReturn(Optional.of(volunteer1));
    when(userInfoUseCase.findById(2)).thenReturn(Optional.of(volunteer2));

    LocalDate date = LocalDate.of( year: 2025, month: 10, dayOfMonth: 15);
    VolunteerAvailableDate volunteer1AvailableDate = new VolunteerAvailableDate( volunteerId: 1, date);
    VolunteerAvailableDate volunteer2AvailableDate = new VolunteerAvailableDate( volunteerId: 2, date);
    when(volunteerAvailabilityRepository.findById(1)).thenReturn(List.of(volunteer1AvailableDate));
    when(volunteerAvailabilityRepository.findById(2)).thenReturn(List.of(volunteer2AvailableDate));

    List<VolunteerAvailableDate> volunteerAvailableDates = new ArrayList<>();
    volunteerAvailableDates.add(volunteer1AvailableDate);
    volunteerAvailableDates.add(volunteer2AvailableDate);

    when(volunteerAvailabilityRepository.findByYearMonth(YearMonth.of( year: 2025, month: 10)))
        .thenReturn(volunteerAvailableDates);

    VisitType visitTypeMock = mock(VisitType.class);
    Visit visit = new Visit(date, volunteer: null, visitTypeMock, List.of(), VisitStatus.PROPOSED);
    when(visitRepository.findAll()).thenReturn(Set.of(visit));
    when(visitTypeRepository.findById(1)).thenReturn(Set.of(visitTypeMock));
    when(visitTypeRepository.findById(2)).thenReturn(Set.of(visitTypeMock));

    service.createVisitPlan();

    assertEquals(VisitStatus.PROPOSED, visit.getVisitStatus());
    assertNotNull(visit.getVolunteer());
    verify(visitRepository).save(visit);
    verify(statePort).setVisitPlanCreated( month: 10, year: 2025, created: true);
}
```

Refactoring

Caso di VisitPlanService

Il refactoring della classe `VisitPlanService` si é reso necessario perché:

- contiene l'algoritmo fondamentale per il funzionamento del programma
- é probabile che possa subire modifiche/aggiunte in futuro
- combina molte informazioni diverse: visite, tipi di visite, date non disponibili, volontari e relative disponibilità

Di seguito i pattern di refactoring applicati

Refactoring

Extract Method - 1

```
List<User> availableVolunteers = new ArrayList<>();

// aggiunge alla lista availableVolunteers tutti i volontari che hanno dato
// disponibilità per il
// giorno 'day'
monthAvailabilities.stream().filter(av -> av.getAvailableDate().equals(day)) //
    // mappa le disponibilità VolunteerAvailableDate con gli oggetti USER volontari
    .map(av -> volunteerById(av.getVolunteerId(), volunteers)) //
    .forEach(availableVolunteers::add);
```

PRIMA

```
List<Integer> availableVolunteerIds = getAvailableVolunteersForVisitDate(visit, volunteerAvailabilitiesNextMonth);
```

DOPO

```
private List<Integer> getAvailableVolunteersForVisitDate(Visit visit, List<VolunteerAvailableDate> volunteerAvailabilitiesNextMonth) {
    return volunteerAvailabilitiesNextMonth.stream() //
        .filter(volunteerAvailableDate -> volunteerAvailableDate.getAvailableDate().equals(visit.getDate())) //
        .map(VolunteerAvailableDate::getVolunteerId) //
        .collect(Collectors.toList());
}
```

Refactoring

Extract Method - 2 + semplificazione logica

PRIMA

```
if (availableVolunteers.isEmpty()) {
    visit.setVisitStatus(VisitStatus.CANCELLED);
    visitRepository.save(visit);
    continue; // passa alla visita successiva
} else if (availableVolunteers.size() == 1) {
    User volunteer = availableVolunteers.getFirst();
    if (canDoVisitType(volunteer, visit.getVisitType().getId()))
        assignVolunteerToVisit(volunteer, visit, monthAvailabilities, volunteerVisitCounts);
    else {
        visit.setVisitStatus(VisitStatus.CANCELLED);
        visitRepository.save(visit);
    }
    continue; // passa alla visita successiva
}

// più di 2 volontari con disponibilità nel giorno considerato
User bestVolunteer = selectBestVolunteer(availableVolunteers, volunteerVisitCounts);
assignVolunteerToVisit(bestVolunteer, visit, monthAvailabilities, volunteerVisitCounts);
```

...

DOPO

```
if (availableVolunteerIds.isEmpty()) {
    setVisitStatusToCancelled(visit);
} else if (availableVolunteerIds.size() == 1) {
    assignAvailableVolunteerToVisitIfPossible(visit, availableVolunteerIds, volunteerAvailabilitiesNextMonth, volunteerIdWithVisitCounts);
} else {
    assignBestVolunteerAvailableToVisit(visit, availableVolunteerIds, volunteerIdWithVisitCounts, volunteerAvailabilitiesNextMonth);
}
}
```

...

Refactoring

Replace temporary with invocation

PRIMA

```
LocalDate today = LocalDate.now(clock);  
YearMonth nextMonth = YearMonth.from(today).plusMonths(1);  
int nextMonthValue = nextMonth.getMonthValue();  
int nextYear = nextMonth.getYear();
```

...

```
removeBlockedDates(nextMonthValue, nextYear);
```

...

```
private void removeBlockedDates(int month, int year) {  
    blockedDatesRepository.loadByMonth(month, year).getDates()
```

```
YearMonth nextMonth = YearMonth.from(LocalDate.now(clock)).plusMonths(1);
```

...

```
removeBlockedDatesFromSelectedMonth(nextMonth);
```

...

```
public void removeBlockedDatesFromSelectedMonth(YearMonth nextMonth) {  
    blockedDatesRepository.loadByMonth(nextMonth.getMonthValue(), nextMonth.getYear())
```

DOPO

Refactoring

Sostituzione di oggetti con relativi id

```
private User selectBestVolunteer(List<User> volunteers, List<VolunteerVisitCount> volunteerVisitCounts) {  
    Comparator<User> volunteerComparator =  
        (v1, v2) -> Integer.compare(visitCountFromVolunteer(v1, volunteerVisitCounts),  
                                     visitCountFromVolunteer(v2, volunteerVisitCounts));  
  
    return volunteers.stream().min(volunteerComparator)  
        .orElseThrow(() -> new IllegalArgumentException("No volunteers available"));  
}
```

PRIMA

```
private int selectBestVolunteerId(List<Integer> volunteerIds, List<VolunteerIdWithVisitCount> volunteerIdWithVisitCounts) {  
    Comparator<Integer> volunteerComparator = //  
        (volunteer1, volunteer2) -> //  
            Integer.compare(visitCountFromVolunteer(volunteer1, volunteerIdWithVisitCounts), //  
                           visitCountFromVolunteer(volunteer2, volunteerIdWithVisitCounts));  
  
    return volunteerIds.stream() //  
        .min(volunteerComparator) //  
        .orElseThrow(() -> new IllegalArgumentException("No volunteers available"));  
}
```

DOPO

Refactoring

PRIMA

```
public void createVisitPlan() {
    if (!canCreateVisitPlan()) {
        throw new IllegalStateException("Non è possibile creare il piano di visita per il mese successivo.");
    }

    LocalDate today = LocalDate.now(clock);
    YearMonth nextMonth = YearMonth.from(today).plusMonths(1);
    int nextMonthValue = nextMonth.getMonthValue();
    int nextYear = nextMonth.getYear();

    List<User> volunteers = userService.getUsersByRole(Role.VOLUNTEER);

    List<VolunteerAvailability> monthAvailabilities =
        initMonthAvailabilities(volunteers, nextMonthValue, nextYear);

    // recupero le visite del prossimo mese e ordino
    List<Visit> monthVisits = initMonthVisits(nextMonthValue, nextYear);

    // Conteggio delle visite assegnate ai volontari nel mese considerato
    // Viene aggiornato ogni visita che viene assegnata una visita a un volontario
    List<VolunteerVisitCount> volunteerVisitCounts = initVolunteerVisitCounts(volunteers);

    for (Visit visit : monthVisits) {
        LocalDate day = visit.getDate();
        List<User> availableVolunteers = new ArrayList<>();

        // aggiunge alla lista availableVolunteers tutti i volontari che hanno date
        // disponibili per il
        // giorno "day"
        monthAvailabilities.stream().filter(av -> av.getAvailabilityDate().equals(day)) //
            // mappa le disponibilità VolunteerAvailability con gli oggetti User volontari
            .map(av -> volunteerById(av.getVolunteerId(), volunteers)) //
            .forEach(availableVolunteers::add);

        if (availableVolunteers.isEmpty()) {
            visit.setVisitStatus(VisitStatus.CANCELLED);
            visitRepository.save(visit);
            continue; // passa alla visita successiva
        } else if (availableVolunteers.size() == 1) {
            User volunteer = availableVolunteers.getFirst();
            if (canDoVisitType(volunteer, visit.getVisitType().getId()))
                assignVolunteerToVisit(volunteer, visit, monthAvailabilities, volunteerVisitCounts);
            else {
                visit.setVisitStatus(VisitStatus.CANCELLED);
                visitRepository.save(visit);
            }
            continue; // passa alla visita successiva
        }

        // più di 2 volontari con disponibilità nel giorno considerato
        User bestVolunteer = selectBestVolunteer(availableVolunteers, volunteerVisitCounts);
        assignVolunteerToVisit(bestVolunteer, visit, monthAvailabilities, volunteerVisitCounts);
    }

    // rimuove le blockedDates del mese considerato
    removeBlockedDates(nextMonthValue, nextYear);
    statePort.setVisitPlanCreated(nextMonthValue, nextYear, true);
}
```

Visione completa del refactor

DOPO

```
public void createVisitPlan() {
    if (!canCreateVisitPlan()) {
        throw new IllegalStateException("Non è possibile creare il piano di visita per il mese successivo.");
    }

    YearMonth nextMonth = YearMonth.from(LocalDate.now(clock)).plusMonths(1);
    List<VolunteerAvailability> volunteerAvailabilitiesNextMonth = volunteerAvailabilityRepository.findbyYearMonth(nextMonth);

    List<Visit> nextMonthSortedVisits = getNextMonthVisitsSorted(nextMonth.getMonthValue(), nextMonth.getYear());

    List<VolunteerIdMonthVisitCount> volunteerIdMonthVisitCounts = initializeVolunteerMonthVisitCounts();

    for (Visit visit : nextMonthSortedVisits) {
        List<Integer> availableVolunteerIds = getAvailableVolunteersForVisit(visit, volunteerAvailabilitiesNextMonth);

        if (availableVolunteerIds.isEmpty()) {
            setVisitStatusToCancelled(visit);
        } else if (availableVolunteerIds.size() == 1) {
            assignAvailableVolunteerToVisitIfPossible(visit, availableVolunteerIds, volunteerAvailabilitiesNextMonth, volunteerIdMonthVisitCounts);
        } else {
            assignBestVolunteerAvailableToVisit(visit, availableVolunteerIds, volunteerIdMonthVisitCounts, volunteerAvailabilitiesNextMonth);
        }
    }

    removeBlockedDatesFromSelectedMonth(nextMonth);
    statePort.setVisitPlanCreated(nextMonth.getMonthValue(), nextMonth.getYear(), true);
}
```

- Da 59 a 27 righe di codice
- Rimosso commenti superflui, sostituendoli con metodi con nomi esplicativi
- Migliorato i nomi delle variabili temporanee, rimuovendole se non strettamente necessarie
- Snellito la logica di calcolo dei dati, assegnando responsabilità a classi più adatte e passando ai metodi solo i parametri necessari